

Laporan PBO Tugas 1

5025241134 Gilbran Mahavikia Raja

September 2, 2025

1 Soal Friendly Number

Friendly numbers

☆ 🔖 🔗 🔍 EasyEnglish ▾

🚩 Found an issue or missing something? Help us improve this problem. Suggest change

🕒 Execution time limit is 1 second

💾 Runtime memory usage limit is 64 megabytes

Two different natural numbers are called friendly if the first one is the sum of divisors of the second number, with the exception of the second number, and the second is the sum of divisors of the first day, except for the very first day. Want to find all the pairs of friendly numbers, both of which belong to the interval from M to N .

Input

In the first row are integers M and N ($1 \leq M \leq N \leq 1\,000\,000$).

Output

In each line of output from a pair of numbers separated by spaces. The first number pair must be less than the second. Rows must be sorted in ascending order of the first of the pair. If the pairs of friendly numbers in the interval not to withdraw "Absent".

Examples

Input #1	Answer #1
200 300	220 284

🕒 Submissions 2K 👤 Acceptance rate 30%

Di soal ini kita diminta mencari pasangan bilangan bersahabat dalam sebuah rentang tertentu. Bilangan bersahabat itu artinya dua bilangan berbeda yang saling "bertukar jumlah faktor pembagi". Maksudnya, kalau kita ambil sebuah bilangan, terus hitung semua pembagi positifnya kecuali dirinya sendiri, hasil penjumlahan itu harus sama dengan bilangan pasangannya. Sebaliknya, kalau kita lakukan hal yang sama untuk bilangan pasangannya, hasilnya harus balik lagi ke bilangan pertama.

Nah, dari rentang $[m, n]$ yang dikasih di input, kita harus cari semua pasangan bilangan yang memenuhi definisi itu. Hasil outputnya ditulis per baris, dengan aturan bilangan yang lebih kecil ditulis dulu, danurut berdasarkan bilangan pertama. Kalau ternyata nggak ada pasangan bilangan bersahabat dalam rentang itu, cukup tulis "Absent"

2 Identifikasi Masalah

2.1 Range

Kita diberi dua bilangan bulat m dan n sebagai batas bawah dan batas atas. Semua perhitungan hanya berlaku untuk bilangan dalam interval $[m, n]$

2.2 Faktor yang Dihitung

Untuk setiap bilangan dalam interval, kita perlu menghitung jumlah faktor sebenarnya, yaitu semua faktor positif bilangan tersebut kecuali dirinya sendiri.

2.3 Pasangan Bilangan yang Dicari

Dari jumlah faktor itu, kita ingin mencari pasangan bilangan (a, b) dengan syarat:

- a dan b ada dalam interval $[m, n]$.
- $a \neq b$
- b adalah jumlah faktor dari a .
- a adalah jumlah faktor dari b .

2.4 Contoh

- Faktor 220 (kecuali dirinya): $1+2+4+5+10+11+20+22+44+55+110 = 284$
- Faktor 284 (kecuali dirinya): $1 + 2 + 4 + 71 + 142 = 220$

3 Solusi

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      ios_base::sync_with_stdio(0);
7      cin.tie(0);
8      cout.tie(0);
9
10     int m, n;
11     cin >> m >> n;
12     vector<int> jumlah_faktor(n + 1, 0);
13
14     for (int i = 1; i <= n / 2; i++){
15         for (int j = 2 * i; j <= n; j += i){
16             jumlah_faktor[j] += i;
17         }
18     }
19
20     bool found = false;
21     for (int a = m; a <= n; a++) {
22         int b = jumlah_faktor[a];
23         if (b > a && b <= n && b >= m){
24             if (jumlah_faktor[b] == a){
25                 cout << a << " " << b << endl;
26                 found = true;
27             }
28         }
29     }
30
31     if (!found) cout << "Absent" << endl;
32     return 0;
33 }
```

Program diawali dengan menyiapkan sebuah array *jumlah_faktor* berukuran $n + 1$. Array ini berfungsi menyimpan hasil penjumlahan faktor untuk setiap bilangan hingga batas n . Dengan begitu, setiap kali program perlu mengetahui jumlah faktor suatu bilangan, hasilnya sudah tersedia tanpa harus dihitung ulang.

Nilai-nilai dalam array diisi menggunakan **teknik sieve**. Pada tahap ini, setiap bilangan i ditambahkan ke seluruh kelipatan j dari $2i$ hingga n . Proses ini memastikan setiap bilangan j akan secara otomatis menerima semua faktor pembaginya. Dengan cara ini, program tidak perlu melakukan perhitungan pembagi secara langsung per bilangan, melainkan mengisi sekaligus untuk semua bilangan dalam satu loop. Inilah yang membuat program jauh lebih efisien, karena kompleksitas waktunya mendekati $O(n \log n)$.

Setelah array terisi, program melakukan iterasi terhadap setiap bilangan a dalam interval $[m, n]$. Untuk setiap a , program menentukan pasangannya $b = \text{jumlah_faktor}[a]$. Pasangan ini hanya valid jika memenuhi tiga syarat: b lebih besar dari a (supaya tidak ada duplikasi), b masih berada dalam interval, dan hasil jumlah faktor dari b kembali sama dengan a . Jika semua syarat terpenuhi, pasangan (a, b) dicetak.

Sebagai tambahan, program menjaga agar tidak terjadi pencetakan ganda dengan aturan $b > a$. Jika setelah semua pemeriksaan tidak ditemukan pasangan yang sesuai, program akan mencetak "Absent". Dengan logika seperti ini, program dapat menangani pencarian bilangan bersahabat dalam interval yang besar secara efisien.

4 Bukti

Berikut adalah bukti bahwa solusi saya mendapatkan verdict **Accepted**.

The screenshot displays a web-based IDE interface for solving the 'Friendly numbers' problem (ID 854). The interface is divided into several sections:

- Header:** Shows the problem title '854 Friendly numbers' and navigation tabs for Statement, Discussion, Attempts, and Submission #f15htau1.
- Accepted Status:** A green checkmark indicates the solution is accepted, dated September 2, 2025 at 12:05:21 PM.
- Test Results:** A table shows the performance of the solution across different test cases:

Test suite	Status	Score	Time	Memory
Test suite #1	Accepted	100/100	25 ms	4.74 MB
Test #1	Accepted	7/7	1 ms	1.10 MB
Test #2	Accepted	7/7	1 ms	1.10 MB
Test #3	Accepted	7/7	3 ms	1.65 MB
- Source code:** The C++ code is displayed in a text editor, showing the implementation of the 'Friendly numbers' algorithm. It includes headers for `<iostream>` and `<vector>`, and uses the `std` namespace. The `main` function reads input `m` and `n`, and calls `jumlah_faktor` to calculate the sum of divisors for each number. The `jumlah_faktor` function is defined as a lambda expression that iterates through divisors of `n` and sums them up.
- Editor:** The C++ code is also shown in a separate editor window, highlighting the logic for finding factors and checking if they are 'friendly'.